PharmaLens: Technical Report on Constraints 02 and 10

Hackathon technical report

1. Executive Summary

PharmaLens is an AI-assisted medication reconciliation platform. A patient uploads a photo of a prescription, and the system extracts the medications and compares them against the hospital's existing record. A pharmacist then reviews anything that differs or looks unreliable.

The architecture has three layers:

- **Vision layer:** Google Gemini Vision 3.8 Flash reads the prescription image and returns structured JSON, constrained by a response schema.
- **Rules layer (FastAPI):** Plain Python re-derives every `Status` and `Sensor Telemetry` value from a Hospital EHR baseline. The model only reads the image. It never decides anything medical.
- **Review layer (Tailwind dashboard):** A pharmacist portal presents conflicts and unreliable readings. Patient-facing records are populated only after a human decision. `localStorage` serves as the mock database.

The principle behind both constraints is that the AI proposes and the pharmacist decides.

2. Constraint 02: Conflicting Data (Stale Records vs. New Input)

The clinical problem

A hospital record from a previous visit may be outdated, and a new prescription may differ from it. Silently trusting either source, or overwriting the old record with the new one, can propagate a dosing error into a patient's medication history.

Backend detection

The server holds a baseline of four medications (`BASELINE_EHR`). After Gemini returns its rows, the `reconcile()` function re-derives each row's status in Python:

- Medicine names are matched case-insensitively against the baseline.

- Doses are compared after `_norm()` strips spacing, casing and punctuation, so `500 mg Twice Daily` equals `500mg twice daily`. Any other difference is a `Conflict`.
- Baseline drugs the model omits are appended as `Not found on prescription`, so an omission can never pass silently.
- Drugs with no baseline entry are labelled `Not on record`.
- Any disagreement between the model's status and the rule-derived status is logged for traceability.

Frontend fallback: no overwriting, explicit provenance

Each medication keeps both readings side by side. The dashboard never replaces the EHR value with the new one.

- **Source labels:** The baseline cell reads `Source: Hospital EHR (Stale: 2023)` and the new cell reads `Source: Patient Upload (New)`.
- **System Alert:** Any prescription with an open stale conflict displays a red banner: *"Conflicting reliable sources detected. Do not silently treat newest records as truth. Manual pharmacist reconciliation required."*
- **Expandable conflict rows:** These explain what changed and offer three actions: **Approve**, **Edit** and **Pass on**.
- **Audit trail:** Every decision stores `decidedAt` and `decidedBy`. Edits preserve `originalExtracted`, so the pre-correction value is never lost.
- **Gated patient record:** The patient's "Current Medications" list and printable record include only pharmacist-reconciled doses. Open conflicts never reach the patient.

3. Constraint 10: Faulty Telemetry (Unreliable OCR Readings)

The AI as a sensor

PharmaLens treats Gemini as an instrument that can drift or misread. Every extracted line carries a confidence value, and the backend applies two independent checks before the dashboard sees it.

Check	Threshold	Backend behaviour
Confidence	**< 90%**	Replaced with a `Low Confidence (n%)` message asking the pharmacist to verify the original image
Dose anomaly	**≥ 5× or ≤ 1/5× baseline** (same unit)	Replaced with a `Sudden Spike Detected` message, with an extra warning if confidence is also low
Missing score	No confidence reported	Marked `Unverified`

Both thresholds (`STABLE_CONFIDENCE = 90` , `SPIKE_RATIO = 5.0`) are named constants. The spike check parses numeric strengths and normalises units (µg, μg and ug become mcg) before comparing. This catches a misplaced decimal such as 5mg read as 50mg.

Isolation and graceful degradation

The frontend maps any flagged reading to a distinct `unreliable` status:

- **Isolation:** The row gets a yellow **Unreliable Sensor Reading** badge and a yellow telemetry cell. This status overrides "Matched", so a shaky reading is never shown as safe even when the doses happen to agree.
- **No auto-approval:** Unreliable rows count as needing review, so the prescription stays in `pending_review` . They are excluded from the patient's current medications and the printable record until a pharmacist approves or edits them.
- **Verification aid:** An eye icon opens the original prescription image beside the extracted value.

The system also fails safely when the sensor itself fails:

- If the model returns malformed or schema-invalid output, the server rejects it with a 502.
- If the server is unreachable, the request times out after 90 seconds, or no medication is readable, the dashboard shows a plain-language error.
- In every failure case **nothing is saved**, so fabricated data never reaches the table.

4. Conclusion

Healthcare software is judged by how it behaves when data is stale or the AI is wrong. PharmaLens treats both cases as expected and routes them to a human. Rule-based verdicts keep every flag explainable, and source labels and timestamped decisions provide an audit trail. Patients see only what a pharmacist has verified. Together these choices make PharmaLens a secure prototype, ready to be hardened for production with a real EHR integration and server-side persistence.